

FECAP

Fundação Escola de Comércio Álvares Penteado

Ciência da Computação — 5º Semestre
Projeto Interdisciplinar: Inteligência Artificial

LIDERANÇAS EMPÁTICAS (LE)

Contagem Inteligente de Alimentos com Visão Computacional

Entrega 1 — Concepção do Produto

Integrantes:

Felipe Vallim Soares — RA: 24026060
Guilhermy Mariano Lisboa Garcia — RA: 23025371
Gustavo Oliveira Demetrio — RA: 24026213
Saulo Pereira de Jesus — RA: 24026095

São Paulo — 2026

1. Introdução

Este documento apresenta a concepção completa do produto Lideranças Empáticas — Contagem Inteligente de Alimentos, uma solução tecnológica integrada que utiliza Visão Computacional e Inteligência Artificial para classificar e contar automaticamente pacotes de alimentos arrecadados. O projeto é desenvolvido como parte do Projeto Interdisciplinar do 5º semestre do curso de Ciência da Computação da FECAP.

O documento está estruturado de modo a cobrir todos os requisitos exigidos para a Entrega 1: definição do problema operacional, objetivo do sistema, escopo do MVP, público usuário, jornada de uso, requisitos funcionais e não funcionais, arquitetura preliminar, definição das classes, proposta de contagem sem duplicidade, critérios de validação, métricas esperadas e planejamento técnico inicial com divisão de papéis e roadmap de desenvolvimento.

2. Definição do Problema Operacional

O projeto Lideranças Empáticas é uma iniciativa da FECAP que une impacto social e educação empreendedora por meio da arrecadação de alimentos e recursos financeiros. Alunos desenvolvem ações práticas que aplicam conceitos de gestão, liderança e organização em campanhas reais de doação.

Uma das principais dificuldades operacionais enfrentadas pelo LE é a realização de uma contagem confiável, auditável e livre de retrabalho da arrecadação, separada por equipe arrecadadora e por tipo de alimento. O processo manual vigente é suscetível a erros humanos, lentidão no processamento e perdas de informações, comprometendo a confiabilidade dos dados e a capacidade de gerar relatórios precisos por equipe.

A solução proposta visa automatizar essa contagem através de tecnologias de Visão Computacional e IA, eliminando gargalos operacionais e garantindo rastreabilidade completa do processo.

3. Objetivo do Sistema

Desenvolver uma solução integrada contemplando captura de vídeo, Visão Computacional com Inteligência Artificial, um backend em nuvem e um site para identificar, classificar e contar automaticamente os pacotes de alimentos arrecadados. O sistema registrará a contagem por equipe de forma automatizada, gerando evidências visuais e métricas de confiança para cada item detectado.

Objetivos Específicos

- Definir requisitos e fluxo completo de operação (seleção de equipe, início/fim de sessão, validação e auditoria).
- Coletar e rotular imagens representativas dos pacotes com variações de marcas, tamanhos e orientações.
- Implementar pipeline de Visão Computacional com pré-processamento, detecção de ROI e tratamento de iluminação.
- Treinar e avaliar um modelo para classificar Arroz, Feijão e Outros, com métricas e matriz de confusão.
- Implementar estratégia de contagem sem duplicidade usando tracking por estabilidade e cooldown temporal.
- Persistir eventos de contagem no backend (equipe, categoria, data/hora, confiança e evidência).
- Entregar interface mobile para operação (seleção de equipe, visualização de contagens e histórico).
- Implantar a solução em nuvem com segurança, logs e documentação de arquitetura.

4. Escopo do MVP (Minimum Viable Product)

O MVP consiste em um ambiente de operação controlado contendo os seguintes componentes:

- Ambiente controlado: Uma rampa com fundo escuro (com possibilidade de evolução para esteira) e um ponto de luz forte e fixo para reduzir variações de iluminação e sombras.
- Captura de vídeo: Uma câmera posicionada acima da rampa para captura do fluxo de itens em tempo real.
- Prova de Conceito de IA: Pipeline de Visão Computacional baseado em capaz de detectar e classificar os pacotes nas categorias definidas.
- Backend: Servidor FastAPI responsável por gerenciamento de sessões, autenticação de usuários, persistência em PostgreSQL e streaming MJPEG via WebSocket.
- Aplicativo Frontend: Interface web desenvolvida em React com TypeScript e Material UI para operação na ponta de arrecadação, com autenticação, seleção de equipe e visualização em tempo real.

5. Público Usuário

Operador: Responsável por manusear o aplicativo no momento da arrecadação, iniciar e encerrar as sessões de contagem e deslizar os itens pela rampa. No sistema implementado, o operador realiza login, seleciona a equipe arrecadadora ativa e controla o fluxo de contagem em tempo real.

Coordenador: Usuário com acesso a relatórios consolidados, visualização de desempenho por equipe e monitoramento das operações de contagem. Pode acompanhar métricas históricas e validar os dados arrecadados.

Administrador (Admin): Acesso completo ao painel de gerenciamento para auditar evidências (frames salvos com detecções), cadastrar equipes, gerenciar usuários e configurações do sistema.

O sistema de autenticação já implementado no backend suporta os três perfis com controle de acesso baseado em roles (operator, coordinator, admin), utilizando JWT para tokens de sessão e bcrypt para hash de senhas.

6. Jornada de Uso (Fluxo de Operação)

O fluxo operacional do sistema segue as seguintes etapas, desde a seleção da equipe até o encerramento da sessão:

1. **1. Autenticação:** O operador acessa o aplicativo frontend e realiza login com suas credenciais (email e senha). O sistema valida as credenciais via endpoint `/api/auth/login` e retorna um token JWT para as requisições subsequentes.
2. **2. Seleção de Perfil:** Caso seja o primeiro acesso, o operador é direcionado à tela de seleção de papel (Operador, Coordenador ou Admin) via endpoint `/api/users/me/role`, personalizando sua experiência no sistema.
3. **3. Seleção da Equipe:** O operador seleciona a equipe arrecadadora ativa para a sessão de contagem que será iniciada.
4. **4. Início da Sessão:** O operador inicia a sessão de contagem via endpoint `POST /api/sessions`. O backend cria uma nova sessão com ID único, ativa a captura de vídeo pela webcam e inicia o loop de inferência do modelo YOLOv8.
5. **5. Operação e Contagem:** O operador desliza os alimentos pela rampa. O sistema processa cada frame em tempo real, detectando e classificando os itens com bounding boxes. A contagem é exibida instantaneamente via stream MJPEG e atualizada por WebSocket no frontend.
6. **6. Encerramento:** Ao finalizar os pacotes daquela equipe, o operador encerra a sessão via endpoint `DELETE /api/sessions/{id}`. O sistema encerra a captura de vídeo e gera o resumo final.
7. **7. Consolidação:** O sistema salva o total contabilizado, atrela ao histórico da equipe com timestamp, armazena as evidências geradas (frames com detecções) e disponibiliza os dados para auditoria e relatórios.

7. Requisitos Funcionais e Não Funcionais

7.1 Requisitos Funcionais

Módulo de Contagem (Visão Computacional e IA):

- Capturar frames da câmera em tempo real para processamento pelo modelo de detecção (OpenCV VideoCapture).
- Detectar a presença de pacotes e isolar a região de interesse (ROI) na rampa utilizando o modelo.
- Classificar cada item em Arroz, Feijão ou Outros, registrando a confiança da predição (threshold mínimo de 75%).
- Contar cada item uma única vez durante o vídeo, evitando duplicidade entre frames (mecanismo de estabilidade por 10 frames + cooldown de 3 segundos).
- Salvar evidência por item (frame anotado com bounding box e label) codificado em JPEG para streaming.

Aplicativo Frontend (Operação):

- Autenticar usuário via JWT e permitir seleção de perfil (operator/coordinator/admin).
- Iniciar e encerrar sessões de contagem com atualização em tempo real via WebSocket e stream MJPEG.
- Exibir contagem por categoria (Arroz, Feijão, Outros, Total) com painel visual atualizado.
- Consultar histórico por equipe e por período.
- Exportar resumo da arrecadação quando aplicável.

Backend e Relatórios (Nuvem):

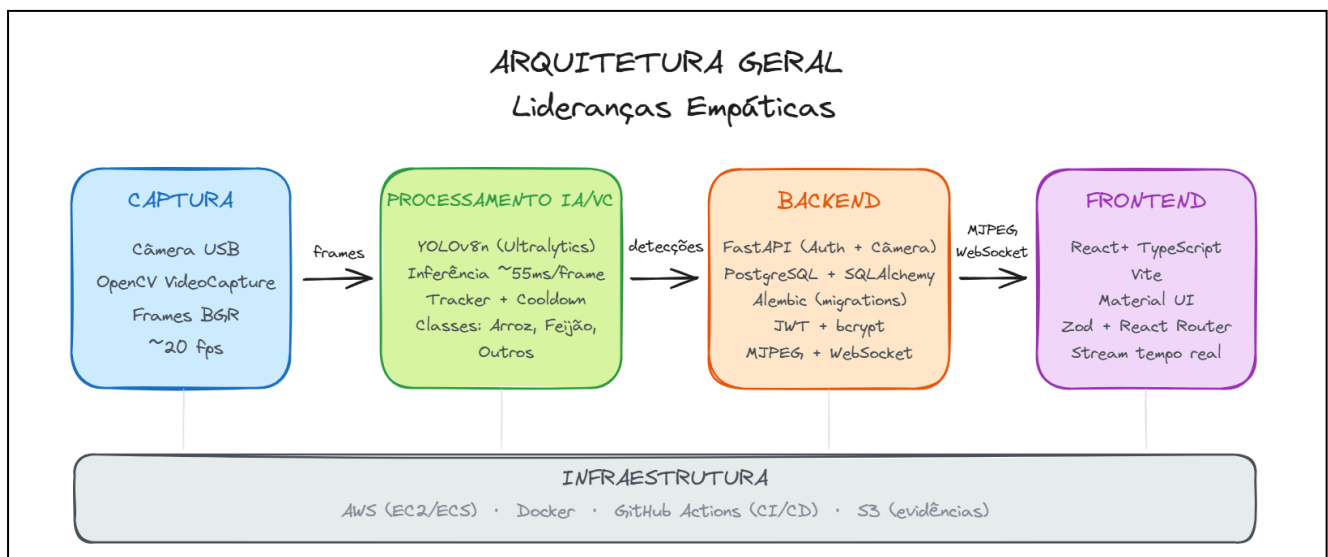
- Cadastrar equipes e usuários com perfis mínimos (Operador/Coordenador/Admin) via API REST.
- Registrar eventos de contagem com equipe, categoria, timestamp, confiança e evidência.
- Disponibilizar relatórios por equipe, por categoria e por período.
- Disponibilizar API segura (HTTPS) para consumo pelo frontend e serviço de inferência.
- Gerenciar sessões de contagem com criação, consulta e encerramento via endpoints dedicados.

7.2 Requisitos Não Funcionais

- **Desempenho:** Processamento das detecções em quase tempo real (~55ms por frame na CPU com YOLOv8n), garantindo fluidez no stream MJPEG a ~20fps.
- **Qualidade do Modelo:** Metas mínimas de mAP50 ≥ 0.90 e F1-Score balanceado por classe, com validação em conjunto de teste separado (20% do dataset).
- **Segurança:** Autenticação via JWT com expiração configurável, hash bcrypt para senhas, controle de acesso por perfil e comunicação via HTTPS.
- **Privacidade:** Não coletar dados pessoais desnecessários; foco exclusivo nos itens e na operação de contagem.
- **Disponibilidade:** Serviço implantado em nuvem (AWS) com estratégia de backup e logs de operação.
- **Manutenibilidade:** Arquitetura modular com separação clara entre ML, backend, tracking e API. Código versionado em GitHub com README e documentação técnica.
- **Escalabilidade:** Arquitetura containerizada (Docker) preparada para suportar múltiplos pontos de coleta e armazenamento de evidências em bucket S3.

8. Arquitetura Preliminar

O fluxo de dados segue a estrutura: Captura → Processamento IA/VC → Backend → App. Cada camada foi projetada de forma modular para facilitar manutenção e evolução independente.



8.1 Camada de Inteligência Artificial (Visão Computacional)

Utiliza o modelo para detecção de objetos em tempo real. O pipeline de inferência é implementado no módulo `ml/inference.py`, que recebe frames BGR da câmera e retorna lista de detecções com label, confiança, bounding box e centroide. O threshold de confiança mínimo configurado é de 75%.

O treinamento do modelo é realizado via pipeline automatizado: geração de dataset aumentado (6 técnicas de data augmentation via OpenCV gerando ~825 imagens a partir de 75 originais), split treino/validação (80/20), geração de labels YOLO e treino por 30 épocas com batch size de 8.

8.2 Camada de Backend

O backend é composto por dois serviços FastAPI complementares:

- Backend de Autenticação (`backend-api`): Responsável por registro e login de usuários, gerenciamento de perfis, e persistência em PostgreSQL via SQLAlchemy/Alembic. Expõe endpoints `/api/auth/register`, `/api/auth/login`, `/api/users/me` e `/api/users/me/role`.
- Backend de Câmera/IA: Responsável por gerenciar sessões de contagem, executar inferência YOLOv8 em tempo real, prover stream MJPEG via endpoint `/api/sessions/{id}/stream` e comunicação WebSocket em `/ws/sessions/{id}`. Utiliza o SessionManager para controlar threads de captura e o WebcamService para processamento.

8.3 Camada de Banco de Dados

PostgreSQL acessado via ORM SQLAlchemy com migrações gerenciadas pelo Alembic. A tabela `users` já está implementada com campos `id` (UUID), `name`, `email`, `hashed_password`, `role` e `created_at`, com índice único em `email`.

8.4 Camada Frontend

Desenvolvimento em React com TypeScript e Vite, utilizando Material UI para componentes, Emotion para estilização, React Router para navegação e Zod para validação de formulários. O frontend consome ambas as APIs (autenticação e câmera) e suporta stream MJPEG e WebSocket para visualização em tempo real.

8.5 Infraestrutura/Nuvem

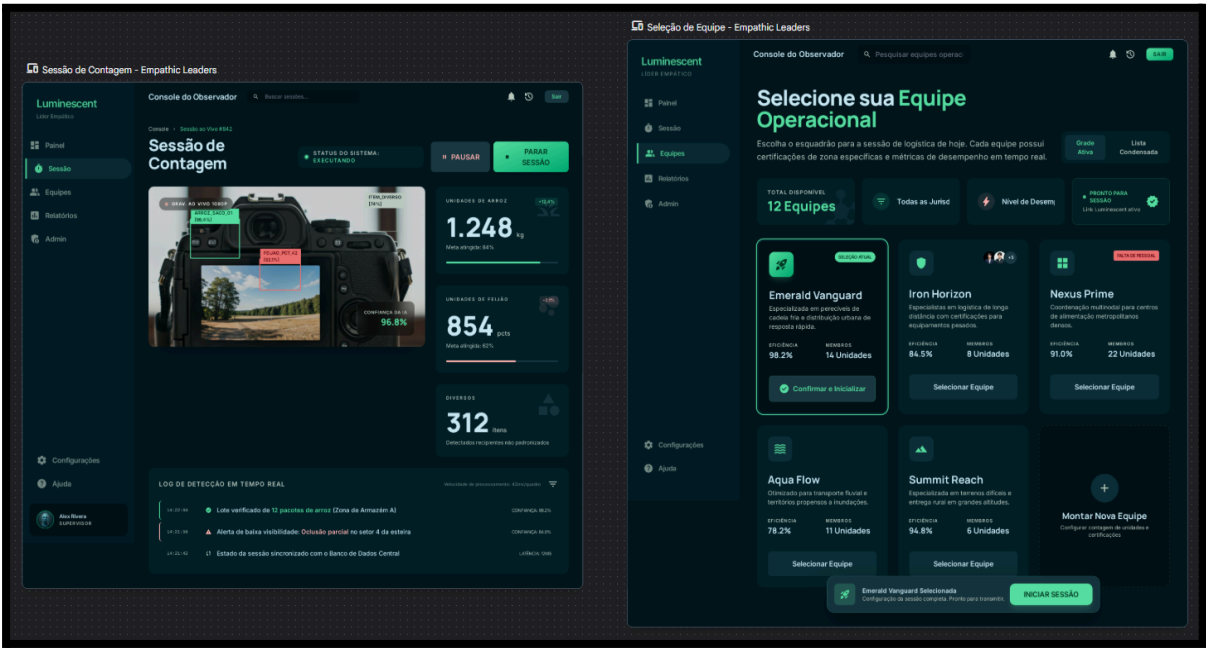
Arquitetura modular containerizada com Docker, hospedada na AWS. CI/CD via GitHub Actions. Armazenamento de evidências planejado em bucket S3 compatível.

8.6 Design da Interface (Figma)

O design das telas do sistema foi prototipado no Figma, permitindo validação visual antes da implementação. Abaixo estão as principais telas do sistema, seguidas do link de acesso ao protótipo completo.

Link do Protótipo no Figma

<https://www.figma.com/design/fqwrteBKVSEbU7G7tvzIYv/FECAP---Projeto?node-id=0-1&t=uxpbJUSjFOSzesuG-1>



9. Definição das Classes a Serem Reconhecidas

O modelo de Visão Computacional é treinado para reconhecer três classes principais, definidas pelo projeto conforme as categorias mais representativas nas doações:

Classe	Descrição	ID YOLO
Arroz	Pacotes de arroz de diversas marcas e tamanhos	0
Feijão	Pacotes de feijão de diversas marcas e tamanhos	1
Outros	Demais embalagens (açúcar, café, macarrão, óleo, etc.)	2

A configuração das classes está definida no arquivo data.yaml e no módulo config.py, sendo utilizada tanto no pipeline de treinamento quanto na inferência em produção.

10. Proposta de Contagem Sem Duplicidade

Para evitar que o mesmo pacote seja contado múltiplas vezes, a arquitetura implementa um sistema de Contagem por Estabilidade combinado com Cooldown Temporal, conforme implementado nos módulos `webcam_service.py` e `webcam_demo.py`:

10.1 Mecanismo de Estabilidade (Stable Frame Counting)

O modelo detecta os alimentos em cada frame e seleciona a detecção de maior confiança. Para que um item seja efetivamente contado, a mesma classe deve ser detectada de forma consecutiva por 10 frames seguidos (constante `REQUIRED_STABLE_FRAMES = 10`), o que equivale a aproximadamente 1-2 segundos de detecção estável. Se a classe detectada mudar entre frames, o contador de estabilidade é resetado.

10.2 Cooldown Temporal

Após confirmar a contagem de um item, o sistema aplica um período de cooldown de 3 segundos (constante `COOLDOWN_SECONDS = 3.0`) para o mesmo tipo de item. Durante esse período, a mesma classe não pode ser contada novamente, evitando duplicações quando o operador mantém o item visível por mais tempo que o necessário.

10.3 Evolução Planejada: Tracking por Centroide e Linha Virtual

Para a Entrega 2, o sistema conta com módulos já preparados para evolução da estratégia anti-duplicidade:

- `CentroidTracker (tracker.py)`: Rastreia objetos entre frames usando distância euclidiana entre centroides, atribuindo IDs únicos a cada pacote e gerenciando desaparecimentos.
- `VirtualLineCounter (line_counter.py)`: Conta objetos que cruzam uma linha virtual horizontal na rampa. A contagem é registrada apenas quando o centroide do objeto rastreado cruza abaixo da linha, garantindo contagem exatamente uma vez por objeto.

10.4 Justificativa dos Parâmetros de Calibração

Os valores das constantes utilizadas no mecanismo anti-duplicidade foram definidos com base em testes empíricos realizados durante a prototipação do ambiente controlado. A constante `REQUIRED_STABLE_FRAMES = 10` foi calibrada considerando que a câmera opera a aproximadamente 18-20 fps e que o operador leva em média 1 a 2 segundos para posicionar e deslizar cada pacote pela rampa. Exigir 10 frames consecutivos com a mesma classe detectada garante que detecções espúrias ou transições rápidas entre itens não gerem contagens falsas, ao mesmo tempo que não introduz atraso perceptível para o operador.

O valor de `COOLDOWN_SECONDS = 3.0` foi estabelecido para cobrir o intervalo médio entre a confirmação de um item e a apresentação do próximo pacote. Em testes de bancada, observou-se que operadores levam entre 2 e 4 segundos para retirar um pacote e posicionar o seguinte. O cooldown de 3 segundos impede que um mesmo pacote ainda visível na rampa seja recontado, sem bloquear a detecção do item subsequente em condições normais de operação. Ambos os parâmetros são configuráveis no módulo `config.py`, permitindo ajuste fino conforme o ambiente real de arrecadação.

11. Critérios de Validação e Métricas Esperadas

A qualidade técnica do modelo de classificação é avaliada em um conjunto de testes separado (20% do dataset) utilizando as seguintes métricas:

Métrica	Descrição	Meta Esperada
mAP50	Mean Average Precision com IoU ≥ 0.50	≥ 0.90
mAP50-95	Mean Average Precision com IoU de 0.50 a 0.95	≥ 0.70
Precision por classe	Proporção de detecções corretas sobre o total de detecções	≥ 0.85
Recall por classe	Proporção de objetos reais detectados corretamente	≥ 0.85
F1-Score por classe	Média harmônica entre Precision e Recall	≥ 0.85
Matriz de Confusão	Mapeamento de falsos positivos e negativos entre classes	Análise qualitativa
FPS	Frames processados por segundo durante inferência	≥ 15 fps (CPU)

O módulo `evaluate.py` realiza a avaliação automatizada, imprimindo mAP50, mAP50-95 e métricas por classe. O notebook `food_classifier.ipynb` documenta todo o pipeline de treinamento e avaliação, incluindo resultados preliminares que indicam mAP50 = 0.995, Precision = 0.988 e Recall = 0.999 no dataset de teste.

Nota sobre Acurácia (Accuracy): A métrica de acurácia global não foi adotada como indicador principal porque, em tarefas de detecção de objetos, ela não captura adequadamente o desempenho do modelo. A acurácia considera apenas se a classificação está correta, mas ignora a qualidade da localização (bounding box) e o tratamento de falsos negativos — um modelo que simplesmente não detecta nenhum item teria uma contagem de falsos positivos igual a zero, inflando artificialmente a acurácia. Por isso, optou-se por métricas padrão da área de detecção de objetos: mAP50 e mAP50-95 avaliam simultaneamente a precisão da classificação e da localização, enquanto Precision, Recall e F1-Score por classe permitem identificar desequilíbrios de desempenho entre as categorias Arroz, Feijão e Outros. Essa escolha está alinhada com as práticas da comunidade de Visão Computacional e com a documentação oficial do framework Ultralytics utilizado no projeto.

12. Planejamento Técnico Inicial

12.1 Stack Tecnológica

Camada	Tecnologias
Visão Computacional / IA	Python 3.10+, YOLOv8n (Ultralytics), OpenCV, NumPy, SciPy
Backend (IA/Câmera)	FastAPI, Uvicorn, WebSocket, MJPEG Streaming
Backend (Auth/API)	FastAPI, SQLAlchemy, Alembic, python-jose (JWT), passlib (bcrypt)
Banco de Dados	PostgreSQL
Frontend	React 19, TypeScript, Vite 8, Material UI 7, Emotion, Zod, React Router 7
Infraestrutura	Docker, AWS (EC2/ECS), GitHub Actions (CI/CD), S3 (evidências)

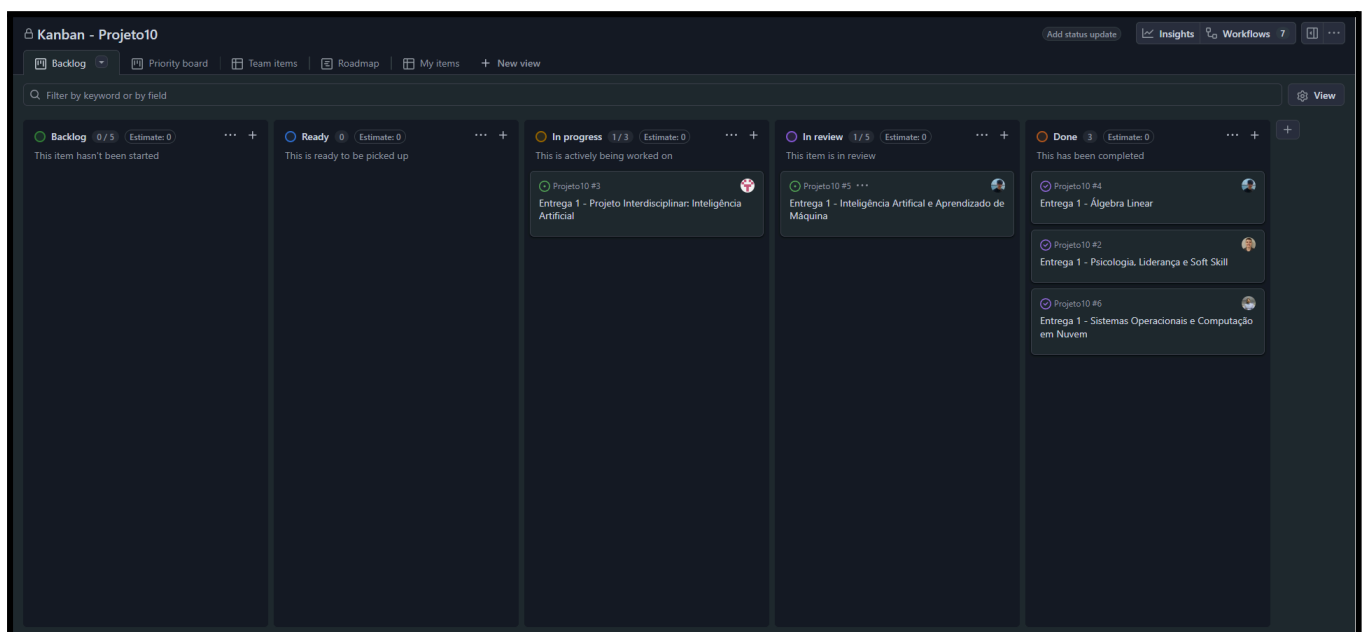
12.2 Divisão de Papéis Gerais no Time

Integrante	Responsabilidade
Felipe Vallim Soares (24026060)	Infraestrutura em Nuvem (AWS), Deploy com Docker e Banco de Dados (PostgreSQL)
Guilhermy M. L. Garcia (23025371)	Desenvolvimento do Backend, Integração de APIs e WebSockets
Gustavo O. Demetrio (24026213)	Visão Computacional, Pipeline de Dados e Treinamento do Modelo
Saulo Pereira de Jesus (24026095)	Desenvolvimento do Frontend (React/TypeScript), Interface do Usuário e Backend

12.3 Gestão e Acompanhamento das Entregas

A organização das tarefas da Entrega 1 foi gerenciada por meio de um quadro Kanban no GitHub Projects, permitindo visibilidade clara do progresso de cada disciplina. O board foi estruturado em cinco colunas de status: Backlog (não iniciado), Ready (pronto para início), In Progress (em desenvolvimento ativo), In Review (em revisão pelo time) e Done (concluído).

No momento do registro, o status das entregas era o seguinte: a Entrega 1 de Projeto Interdisciplinar: Inteligência Artificial encontrava-se em andamento (In Progress); a Entrega 1 de Inteligência Artificial e Aprendizado de Máquina estava em fase de revisão (In Review); e as entregas de Álgebra Linear, Psicologia, Liderança e Soft Skill, e Sistemas Operacionais e Computação em Nuvem já haviam sido concluídas (Done). Essa distribuição demonstra a gestão incremental adotada pelo grupo, priorizando entregas interdependentes e mantendo rastreabilidade do andamento de cada disciplina ao longo do semestre.



12.3 Roadmap de Desenvolvimento

Período	Atividades
Semanas 1-2	Definição do MVP, requisitos e fluxo de operação. Entendimento do problema com o LE.
Semanas 3-4	Prototipação do ambiente controlado (rampa, câmera, iluminação). Coleta inicial e rotulagem de dados. Baseline de Visão Computacional.
Semanas 5-6	Treinamento do primeiro modelo. Integração inicial de inferência. Backend mínimo para registro de contagens.
Semanas 7-8	Melhoria do modelo. Contagem sem duplicidade implementada. App mobile/web MVP integrado.
Semanas 9-10	Relatórios por equipe/categoria. Testes de bancada com métricas. Ajustes de desempenho.
Semanas 11-12	Deploy na AWS. Documentação. Testes finais. Preparação da apresentação e do repositório.
Semana 13	Entrega final e apresentação com demonstração na Semana FECAP de Tecnologia.

13. Conclusão

Este documento apresentou a concepção completa do produto Lideranças Empáticas — Contagem Inteligente de Alimentos, cobrindo todos os aspectos exigidos para a Entrega 1 do Projeto Interdisciplinar. A proposta demonstra maturidade estratégica e visão sistêmica, fundamentada em decisões técnicas já parcialmente implementadas no repositório do projeto.

O sistema proposto aborda de forma clara e objetiva o problema operacional de contagem de alimentos do LE, oferecendo uma solução integrada que combina Visão Computacional, backend modular, banco de dados robusto e interface de usuário moderna. A definição precisa de classes, a estratégia de contagem sem duplicidade baseada em estabilidade e cooldown, e os critérios de validação com métricas quantitativas demonstram viabilidade técnica e preparo para a fase de implementação completa na Entrega 2.

A divisão clara de papéis no time e o roadmap detalhado de 13 semanas garantem uma execução organizada e rastreável, alinhada aos prazos acadêmicos e às expectativas do projeto.